

SQL Practice Exam

Questions with Submitted Solutions

Question 1: Days Between Customer Orders

Database: CUSTOMERS_ORDERS_DB

Problem Statement

For each order placed by a customer, calculate the number of days that have passed since the customer's immediately previous order. Use the LAG() window function to retrieve the previous order date for the same customer.

- Process the orders separately for each customer using customer_id.
- Use LAG() to retrieve the immediately previous order_date for each customer.
- Determine the sequence of orders using order_date and then order_id.
- For the first order of each customer, days_since_previous_order should be NULL.
- Sort by customer_name, order_date, order_id.

Required Output Columns: customer_name, order_id, order_date, days_since_previous_order

Submitted Solution — Attempt 1

```
SELECT C.customer_id,  
O.order_id,  
O.order_date,  
DATEDIFF(O.order_date,LAG(O.order_date) OVER (PARTITION BY O.customer_id  
order by O.order_date)) as days_since_previous_order  
FROM Orders O  
JOIN Customers C  
ON O.customer_id = C.customer_id  
ORDER BY c.customer_name,o.order_date,o.order_id;
```

Note: This version selects customer_id instead of the required customer_name column, and the LAG() ORDER BY does not include order_id as a tiebreaker.

Submitted Solution — Attempt 2 (Corrected)

```
SELECT  
c.customer_name,  
o.order_id,  
o.order_date,  
DATEDIFF(  
o.order_date,  
LAG(o.order_date) OVER (PARTITION BY o.customer_id ORDER BY  
o.order_date,o.order_id)  
) AS days_since_previous_order  
FROM Orders o  
JOIN Customers c ON c.customer_id= o.customer_id  
ORDER BY  
c.customer_name,  
o.order_date,  
o.order_id;
```

This version correctly selects customer_name and includes order_id as a tiebreaker in LAG()'s ORDER BY. This is the correct solution.

Question 2: Event Average Finish Time

Database: ATHLETICS_RESULTS_DB

Problem Statement

For every recorded result, display the event name, athlete name, finish time, and the average finish time of all recorded performances within the same event. Use AVG() with PARTITION BY. Only status = 'Recorded' should be included.

Required Output Columns: event_name, athlete_name, finish_time, avg_event_time

Sort by: event_name, finish_time, athlete_name

Submitted Solution

```
SELECT E.event_name,  
A.athlete_name,  
R.finish_time,  
ROUND(AVG(R.finish_time) OVER (PARTITION BY E.event_name),2) as  
avg_event_time  
FROM RESULTS R  
JOIN EVENTS E  
USING(event_id)  
JOIN Athletes A  
USING(athlete_id)  
WHERE R.status = 'Recorded'
```

Note: This query is missing the final ORDER BY event_name, finish_time, athlete_name — without it, row order isn't guaranteed to match the required sort. It also partitions by event_name rather than event_id, which works here only because event_name happens to be unique per event.

Question 3: Average Delivery Time by Route

Database: DELIVERY_TRACKING_DB

Problem Statement

For every completed delivery, show the route name, driver name, delivery time, and the average delivery time of all completed deliveries within the same route. Use AVG() with PARTITION BY route_id. Only status = 'Completed' should be included.

Required Output Columns: route_name, driver_name, delivery_time, avg_route_time

Sort by: route_name, delivery_time, driver_name

Submitted Solution

```
SELECT  
route_name,  
driver_name,  
delivery_time,  
avg_route_time  
FROM(
```

```

SELECT R.route_name,
D.driver_name,
D1.delivery_time,
ROUND(AVG(D1.delivery_time) OVER (PARTITION BY R.route_name),2) as
avg_route_time
FROM Routes R
JOIN Deliveries D1
ON R.route_id = D1.route_id
JOIN Drivers D
ON D.driver_id = D1.driver_id
WHERE D1.status ='Completed') AS D
ORDER BY route_name,delivery_time,driver_name

```

This is correct and includes the required ORDER BY. Minor style note: the derived table alias "D" reuses the same letter as the Drivers table alias used inside the subquery — it works because the inner alias goes out of scope once the subquery closes, but a distinct name (e.g. ranked) would be clearer.

Question 4: Days Between Patient Appointments

Database: PATIENT_APPOINTMENTS_DB

Problem Statement

For each appointment, calculate the number of days that have passed since the patient's immediately previous appointment. Use LAG() partitioned per patient, ordered by appointment_date then appointment_id.

Required Output Columns: patient_name, appointment_id, appointment_date, days_since_previous_appointment

Sort by: patient_name, appointment_date, appointment_id

Submitted Solution

```

SELECT P.patient_name,
A.appointment_id,
A.appointment_date,
DATEDIFF(A.appointment_date,LAG(A.appointment_date) OVER (PARTITION BY
P.patient_name ORDER BY A.appointment_date)) as
days_since_previous_appointment
FROM Patients P
JOIN Appointments A
ON P.patient_id = A.patient_id
ORDER BY
    P.patient_name,
    A.appointment_date,
    A.appointment_id;

```

Note: The join condition is now correct (patient_id = patient_id). Two remaining issues: it partitions by patient_name instead of patient_id (risky if two patients share a name), and the window's ORDER BY is missing appointment_id as a tiebreaker for same-date appointments.